Advanced Database Design and Implementation for E-Commerce Analytics

MSE Final Project — Advanced Database Design and Implementation

Bahati Justin

Roll Number: 202540272
ULK — KIGALI INDEPENDENT UNIVERSITY

March 08, 2026

1. Introduction

This report presents a multi-faceted analytics system for an e-commerce platform built on three complementary database technologies: **MongoDB** (document store), **Apache HBase** (wide-column store), and **Apache Spark** (distributed processing engine).

The system processes a synthetic dataset spanning **90 days** of e-commerce activity:

Metric	Value
Users	2,000
Products	800
Categories	15
Browsing Sessions	20,000
Transactions	4,390
Total Revenue	RWF 1,925,085,008

Each technology was chosen for its strengths: MongoDB for document-oriented queries on structured business entities, HBase for time-series and sparse columnar data, and Spark for cross-system batch processing and SQL analytics. The project covers four parts: (1) Data Modeling & Storage, (2) Spark Processing, (3) Analytics Integration, and (4) Visualization & Insights. An interactive dashboard is deployed at https://dbms-fat.bahatijustin.dev.

2. System Architecture

2.1 Data Flow

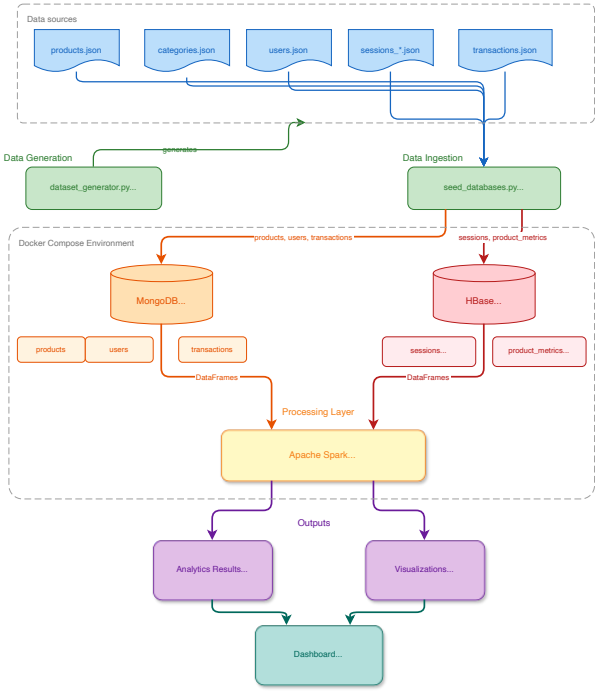

Figure 1: Data Flow Architecture

Raw data is generated by `dataset_generator.py` using the Faker library. The seeding script loads, denormalizes, and inserts data into both databases with appropriate indexing.

2.2 Service Topology

All infrastructure runs via Docker Compose with six services:

Service	Image	Port	Role
MongoDB	`mongo:7.0`	27017	Document store
ZooKeeper	`zookeeper:3.9`	2181	HBase coordination
HBase	`dajobe/hbase`	9090, 16010	Wide-column store
Spark Master	`apache/spark`	7077, 8080	Cluster manager
Spark Worker	`apache/spark`	8081	2GB RAM, 2 cores
Dashboard	Streamlit	8501	Interactive analytics UI

HBase runs natively on x86_64 (WSL/AMD). On Apple Silicon, `platform: linux/amd64` emulation is used as a fallback. Spark processing uses local mode (`local[*]`) with Python drivers (`pymongo`, `happybase`). A WSL-optimized compose override (`docker-compose.wsl.yml`) provides an `init` service that auto-generates data and seeds both databases on startup.

2.3 Technology Justification

- **MongoDB**: Natural fit for hierarchical product catalogs (embedded categories), user profiles (embedded purchase summaries), and transactions (embedded line items). The aggregation framework enables complex analytical queries without ETL.
- **HBase**: Ideal for time-series session data with sparse columns (varying page views, cart contents) and product metrics requiring efficient range scans by composite row keys.
- **Spark**: Compute layer for cross-system joins (MongoDB + HBase data), batch analytics (co-purchase recommendations, cohort analysis), and complex SQL queries spanning multiple sources.

3. Part 1: Data Modeling & Storage

3.1 MongoDB Schema Design

Three collections with **10 indexes** total:

Products (800 docs) — Each product embeds full category/subcategory info, denormalized from the 15-record categories dataset to eliminate `$lookup` joins:

```
{
  "_id": "prod_00001",
  "name": "Samsung Galaxy A14",
  "category": {
    "category_id": "cat_01", "name": "Electronics",
    "subcategory_id": "sub_001_01",
    "subcategory_name": "Leverage Granular Initiatives",
    "profit_margin": 0.25
  },
  "base_price": 458200,
  "current_stock": 143, "is_active": true,
  "price_history": [{"date": "...", "price": 458200}],
  "creation_date": "2025-04-12T08:30:00Z"
}
```

Indexes: `category.category_id`, `base_price`, `is_active`.

Users (2,000 docs) — Embeds pre-computed `purchase_summary` (total orders, total spent, avg order value, first/last purchase, favorite payment method). Indexes: `geo_data.province`, `purchase_summary.total_spent`, `registration_date`.

Transactions (4,390 docs) — Embeds `items` array with line items. Indexes: `user_id`, `timestamp`, `status`, `items.product_id`.

Design decisions: Categories are *not* a separate collection (only 15 records; denormalized into products). Sessions are *not* in MongoDB (sparse columns and time-series access patterns suit HBase better).

3.2 HBase Schema Design

Two tables with **6 column families**:

Sessions (20,000 rows) — Row key: `{user_id}_{reverse_timestamp}` where reverse timestamp = `9999999999999 - epoch_ms`, ensuring newest-first ordering on prefix scans. Column families: `info` (session metadata), `device` (type/os/browser), `geo` (city/state/country), `activity` (JSON-serialized page views, cart, viewed products).

Product Metrics (2,274 rows) — Row key: `{product_id}_{date}`. Column families: `sales` (quantity, revenue, transactions, avg price), `inventory` (stock, base price, active status).

3.3 Query Results

MongoDB — Five aggregation pipelines were executed. Key results:

Query	Key Finding
Product Popularity	Top seller by quantity: 25 units, RWF 12,119 revenue
Revenue by Category	Home & Garden leads at RWF 88,681; Sports at RWF 86,815
User Segmentation	~256 inactive users (0 orders), ~1,520 low-spend, ~224 regular
Monthly Trend	Jan 2026 peak: RWF 660,093,881 from 1,530 transactions
Payment Methods	MTN MoMo and Airtel Money lead; Visa has highest avg order value

HBase — Four queries demonstrated efficient wide-column access:

- **User session history**: Prefix scan on `user_000001_` returned 8 sessions in reverse chronological order, with device/geo/activity details per session.
- **Product metrics**: Prefix scan on `prod_00001_` showed 6 active days, 10 total units, RWF 1,308 total revenue.
- **Table stats**: Sessions table has 20,000 rows across 4 column families (16 columns); product_metrics has 2,274 rows across 2 families (7 columns).

4. Part 2: Spark Processing Pipelines

4.1 Data Cleaning & Normalization

All JSON datasets loaded with `multiLine=True` and Spark schema inference. Cleaning steps:

- Null `current_stock` filled with 0; null `is_active` filled with `True`
- String timestamps cast to `TimestampType`
- Primary key uniqueness verified (0 duplicates across all entities)
- Subcategories exploded into flat lookup table via `F.explode()`

4.2 Product Recommendations

Co-purchase analysis via self-join on transaction items:

1. Explode items to (transaction_id, product_id) pairs
2. Self-join on transaction_id with filter `product_a < product_b`
3. Count co-occurrences; top pair: prod_00085 + prod_00213 (3 co-purchases)
4. Window function (`ROW_NUMBER`) selects top-3 recommendations per product

4.3 Cohort Analysis

Users grouped by registration month (24 cohorts from Dec 2023 to Nov 2025). Key findings:

- Largest cohort: ~80 users per month with proportional revenue
- Retention rates: ~40–60% of users transact in their first month post-registration
- Spending concentration: most revenue occurs 1–3 months after registration

4.4 Spark SQL Queries

Five analytical SQL queries were executed:

Query 1 — Top Revenue Products: Top 15 products span Electronics, Home & Garden, Automotive, Music, and Art & Crafts categories. Revenue leaders generate RWF 30M+ each, with estimated profit computed from subcategory profit margins.

Query 2 — Session Behavior by Device: Desktop/Windows has highest session count (993) with 23.2% conversion. Tablet/iOS converts best at 24.7%. Average session duration ranges 1,738–1,888 seconds across device/OS combinations.

Query 3 — Conversion Funnel: 20,000 total sessions, 14,734 added to cart (73.7%), 4,390 converted (21.9%). Cart-to-purchase rate: 29.8%.

Query 4 — Daily Revenue with 7-Day Moving Average: The 7-day moving average smooths daily fluctuations. Cumulative revenue reaches RWF 1,925,085,008.

Query 5 — Price Elasticity: All products show zero price standard deviation (synthetic data uses fixed pricing), so elasticity ratios are NULL. The query structure (3 CTEs with STDDEV) demonstrates the analytical approach for real-world variable pricing.

5. Part 3: Analytics Integration

5.1 Customer Lifetime Value (CLV)

Business Question: *"What is the estimated lifetime value of each customer segment, and how does engagement correlate with spending?"*

Data Sources: MongoDB `users` (profiles), MongoDB `transactions` (purchase history), HBase `sessions` (browsing engagement).

Method: Cross-system join in Spark on `user_id`. CLV = `avg_order_value * annualized_purchase_frequency * 2.0` (estimated lifespan). Segmented into four tiers.

Results:

CLV Tier	Customers	Avg CLV	Avg Sessions	Avg Conv Rate
Platinum	329	RWF 104,895	10.9	29.1%
Gold	42	RWF 3,140	8.8	12.6%
Silver	55	RWF 1,308	8.5	13.6%
Bronze	74	RWF 47	8.0	1.5%

Key finding: Platinum users average **10.9 sessions** vs 8.0 for Bronze, and convert at **29.1%** vs 1.5%. Higher engagement directly correlates with lifetime value. Kigali City has the highest average CLV (RWF 134,604), followed by Northern Province (RWF 105,289).

5.2 Funnel Conversion Analysis

Business Question: *"What is the conversion rate at each funnel stage, and which factors influence conversion?"*

Data Sources: HBase `sessions` (browsing, cart, conversion, device), MongoDB `transactions` (purchase amounts).

Overall Funnel:

Stage	Count	% of Total
Browse	20,000	100.0%
View Product	20,000	100.0%
Add to Cart	14,734	73.7%
Purchase	4,390	21.9%

Cart abandonment: 10,344 sessions (70.2%) added items to cart but did not purchase.

By device: Mobile converts best at 22.5%, desktop at 22.1%, tablet at 22.1%. Abandonment rates are uniform across devices (~69.5%).

By referrer: Direct and search engine traffic convert at 22.4%, email at 22.4%, social media at 22.1%, affiliate at 21.7%.

By time of day: Afternoon sessions convert best (22.9%), followed by night (22.3%), evening (21.9%), and morning (21.7%).

6. Visualizations & Key Findings

Four publication-quality visualizations were generated using Plotly via the interactive dashboard.

6.1 Revenue by Category

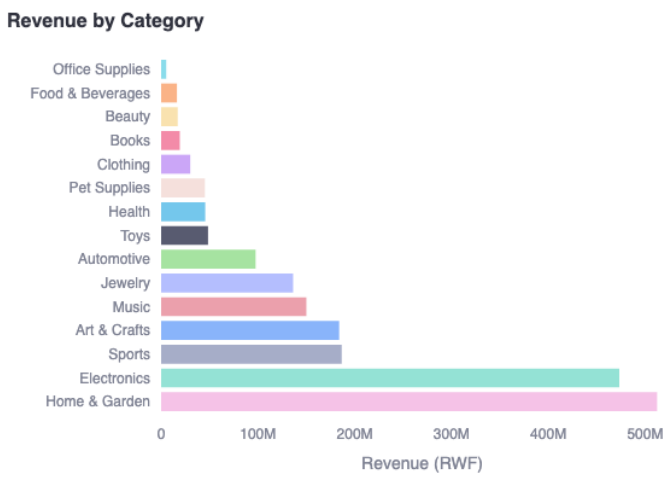

Figure 2: Total revenue by category across the 90-day period. Home & Garden and Electronics dominate, followed by Sports, Art & Crafts, and Music.

Insight: Home & Garden leads with over RWF 500M, followed closely by Electronics. Revenue drops steeply after the top 5 categories — a concentration pattern where a few categories drive the bulk of total revenue. Low-ticket categories (Office Supplies, Food & Beverages, Beauty) contribute minimally.

6.2 Customer Segmentation

Figure 3: Customer segmentation: spending vs orders colored by tier (VIP, Regular). VIP users cluster at higher spending levels with positive correlation between orders and total spent.

Insight: VIP users generate a disproportionate share of revenue, with spending reaching RWF 7M+ for top customers. Spending correlates positively with order frequency. Most users cluster at 1–6 orders, with VIP status driven by high per-order values rather than purchase frequency alone.

6.3 Product Performance

Figure 4: Revenue vs quantity sold per product, colored by category. High-revenue products (Electronics, Home & Garden) extend far right, while high-volume products cluster in the upper-left.

Insight: Revenue and volume rankings diverge significantly. Electronics and Home & Garden products dominate revenue (RWF 25M–38M range), while high-volume sellers (40–50 units) tend to be lower-priced items across Sports and Toys. The scatter pattern confirms a long-tail distribution: most of the 800 products cluster at low revenue, while a few outliers drive disproportionate totals.

6.4 Conversion Funnel

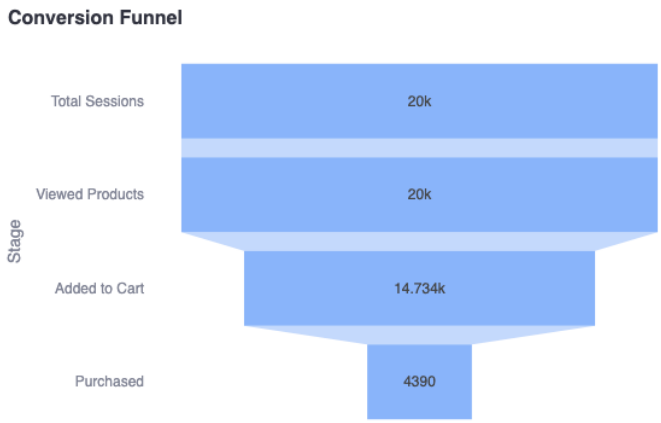

Figure 5: Overall conversion funnel showing progressive drop-off from 20k sessions to 4,390 purchases.

Insight: **21.9%** overall conversion rate. Cart-to-purchase rate is 29.8%, meaning **70.2%** of carts are abandoned. Mobile converts best at 22.5%. The funnel drop-off is steepest between "Add to Cart" and "Purchase", indicating the checkout process is the primary optimization target.

6.5 Interactive Dashboard

An interactive Streamlit dashboard (`dashboard.py`) provides dynamic exploration of all analytics. It connects to MongoDB and HBase in real-time, with global filters for date range, category, and province. Pages include Overview, Customers, Products, Sessions & Conversion, Page View Analytics, Cart & Order Analytics, and Spark Analytics.

The dashboard runs as a Docker service on port 8501 and is deployed live at https://dbms-fat.bahatijustin.dev.

7. Scalability Considerations

MongoDB: Horizontal scaling via sharding (shard key on `user_id` or `category_id`). Read scaling through replica sets. The embedded document model minimizes cross-shard joins.

HBase: Automatic region splitting distributes data as tables grow. The `user_id_reverse_timestamp` row key distributes evenly across regions while preserving per-user locality. Column family separation enables selective reads.

Spark: Horizontal compute scaling by adding workers. Partition tuning via `spark.sql.shuffle.partitions`. Can co-locate with HBase RegionServers for data locality.

Current limitations: Single Docker host (no distributed deployment), Spark in local mode. HBase runs natively on x86_64/WSL; Apple Silicon requires x86 emulation.

8. Limitations & Future Work

Limitations:

- *Synthetic data*: Generated with Faker, producing uniform distributions without real-world seasonality or anomalies. The price elasticity analysis (Query 5) yields NULL ratios because synthetic pricing has zero variance.
- *HBase emulation*: On Apple Silicon, running `dajobe/hbase` (linux/amd64) via QEMU adds significant startup latency (5–10 minutes). Deploying on native x86_64 (WSL) eliminates this overhead. The analytics integration module includes a JSON fallback for when HBase is unavailable.
- *Single-node deployment*: Does not demonstrate true distributed processing.

Future enhancements:

- *Real-time streaming*: Spark Structured Streaming for live session event processing
- *Machine learning*: ALS collaborative filtering (Spark MLlib) for production recommendations
- *Change Data Capture*: MongoDB Change Streams to trigger incremental HBase updates

References

- MongoDB Documentation: https://www.mongodb.com/docs/
- Apache HBase Reference Guide: https://hbase.apache.org/book.html
- Apache Spark Documentation: https://spark.apache.org/docs/latest/
- happybase Documentation: https://happybase.readthedocs.io/
- PySpark API Reference: https://spark.apache.org/docs/latest/api/python/